

TECHNICAL IMPLEMENTATION REPORT & ROADMAP

RFT Platform Administrator Portal

A comprehensive technical guide outlining the directory structure, implemented modules, role-based user interfaces, and the core functional architecture of the Next.js institutional management portal. Contains a detailed analysis of what is currently live and the future roadmap for production.

IMPLEMENTATION SCOPE:

- * Super Administrator Portal - Global Network Control
- * School Administrator Portal - Campus-Level Academic Operations
- * Lecturer Workspace - Course, Session, and Assessment Operations

DOCUMENT METADATA

Author: Core Architecture Team
Version: 1.0.0 (Release-Ready)
Date: June 2026

SYSTEM STACK INFO

Framework: Next.js 16.2.2 (App Router)
Runtime: React 19.2.4 / TypeScript 5.7
Styling: Tailwind CSS 3.4.17

1. Executive Summary & Tech Stack

Executive Summary

The RFT Platform Administrator Portal is a highly customized, web-based management suite designed to handle the hierarchical academic administration of universities, faculties, and courses. The portal is structured around three primary roles (Super Administrator, School Administrator, and Lecturer) allowing granular operations from global multi-campus billing and onboarding down to individual class scheduling and student representative coordination.

This document lists the architectural structure of the codebase, details the design systems and modules currently implemented in the demo environment, and provides a clear technical roadmap outlining the necessary back-end integrations and security measures left to be implemented for full production deployment.

Core Technology Stack

The application is built using modern, stable web standards to guarantee fast performance, reusability, and layout stability. The technical stack comprises:

- **Next.js 16 (App Router)** Used for handling directory-based routing, layout inheritance, and optimized bundle sizes.
- **React 19 & TypeScript 5** Ensures strict type-safety across components, custom hooks, and platform schemas.
- **Tailwind CSS 3** Powers the visual design system, layout boundaries, responsive breakpoints, and animations.
- **Recharts** A composable charting library integrated into the lecturer and administration dashboards to visualize grading trends.
- **Lucide React** A standard icon package for visual iconography matching the styling structure.

2. Codebase Architecture & Filesystem

The repository follows a clean Next.js App Router structure where routes are organized in groups, and modular logic is isolated in directory containers under components, hooks, lib, and types. Below is an overview of the directory structure:

Path / Directory	Role & Technical Purpose
/app	Root directory for routes, layouts, global styles, and page entrypoints.
(auth)/login	Authentication page rendering the login interface and user selection.
(dashboard)/layout.tsx	Dashboard layout shell inherited by all administrator roles.
(dashboard)/dashboard/super	Super Administrator workspaces (schools, onboarding, fees, roles, analytics).
(dashboard)/dashboard/school	School Administrator workspaces (faculties, courses, levels, lecturers, students).
(dashboard)/dashboard/lecturer	Lecturer workspaces (enrollment, class-reps, content, assessments, schedule).
/components	Modular UI system, dashboard elements, and layout containers.
/auth	LoginForm and MockCredentials listing.
/dashboard	Reusable admin elements like FacultyTable, StatsCard, ContentUploadPanel.
/layout	Core layout containers: Sidebar, MobileSidebar, Topbar, and Logo markers.
/lecturer	Lecturer components: AssignedCoursesTable, PerformanceChart, ClassRepsPanel.
/ui	Atomic UI components: MetricCard, SimpleTable, FormField, Modal, AppButton, StatusBadge.
/demo	Functional onboarding, queue widgets, and assessment constructor blocks.
/hooks	Custom client-side hooks: useAuth (session management) and useSidebar.
/lib	Mock database schemas (mock-users.ts) and platform configuration (navigation.ts).
/types	Global TypeScript interface files and role enumerations.

State Management & Helper Utilities

All client-side states such as sidebar collapsible behavior, authentication parameters, and visual toast alerts are handled using React context providers (e.g., ToastProvider.tsx) and local storage cache updates. Mock database values are imported as static JSON objects to render realistic records and metrics during demonstrations, maintaining complete separation between the UI and mock layers.

3. Implemented Modules: Auth & Super Admin

The following modules have been fully implemented, styled with custom styling (dark-blue/gold/glassmorphism), and populated with functional demo data. They are live and ready for production API integration.

Authentication & Session Guards

The authentication interface (/login) provides credential inputs, mock password toggling, and a 'Mock Accounts' quick-select panel (LoginForm.tsx). A custom hook (useAuth.ts) reads mock-users.ts and performs credentials validation. Upon successful authentication, cookies are set ('rft_user', 'rft_role') and the user is redirected to the role-specific route container. Front-end protection blocks access to dashboard sub-routes if the user session cookie is missing.

Super Administrator Portal (/dashboard/super/*)

The Super Admin role supervises the global network of universities, manages pricing plans, authorizes accounts, and reviews general metrics. Key sub-modules include:

- **Global Dashboard** Displays overview cards (Total Schools: 24, Lecturers: 1,204, Active Students: 48,921) and school overview tables.
- **School Directory** A dynamic list (/super/schools) showing capacity metrics, location pins, and current plans with custom Action Menus.
- **Onboarding Workspace** A detailed, multi-section form (/super/onboarding) supporting general details, admin credentials, and tier-specific setup fee calculations.
- **Subscription & Fees Workspace** A complete billing workspace (/super/fees) showcasing Tier Cards (Free, Premium, Enterprise), recent transactional tables, and billing stats.
- **User Access & Roles** An access provisioning dashboard (/super/roles) for mapping Full Names, Staff IDs, and system roles directly to specific university nodes.
- **Global Analytics** Summarizes network metrics and provides visual distribution grids.

4. Implemented Modules: School Admin & Lecturer

School Administrator Portal (/dashboard/school/*)

The School Admin role administers campus-level records, manages departmental faculties, coordinates teaching staff, and oversees student intakes. Main modules include:

- **Metrics Overview**Key KPIs (Students: 12,482, Lecturers: 845, Active Courses: 216) and distribution buttons for academic levels.
- **Faculty Operations**A loading grid (/school/faculties) detailing lecturer allocations, student enrollment pressure, and academic health status metrics.
- **Level Management**Provides detailed progress summaries from 100L up to 400L.
- **Course Portfolio**A structured table (/school/courses) mapping Course Codes, Levels, Type (Core vs Advanced), and Class Rep designations.
- **Lecturer Directory**A directory (/school/lecturers) for verifying and adding teaching staff.
- **Students & Class Reps Workspace**Combines intake verification queues and communication lead assignment panels.
- **Institutional Content Library**A repository (/school/content) for sharing syllabus guidelines, policies, and campus notice packs.

Lecturer Workspace (/dashboard/lecturer/*)

The Lecturer workspace supports day-to-day academic operations, class scheduling, and grading assessments. Main modules include:

- **Course Dashboard**Displays assigned courses (GST101, LAW302, etc.), pending actions, and mid-term performance trends.
- **Recharts Performance Analytics**A bar chart displaying average grades across courses, visual ranking, and pending reviews.
- **Class Scheduling Form**A constructor (/lecturer/schedule) for creating sessions, setting venue parameters, and configuring audience notifications.
- **Assessment Builder**A setup panel (/lecturer/assessments) for configuring quizzes, assignments, and exam formats (Timed CBT, Portal Upload).
- **Student Enrollment & Class Reps**Views for class representative coordination and monitoring active registrations.

5. Technical Gaps & Future Roadmap

To transform the current front-end portal into a secure, production-grade application, the following gaps must be addressed in the next phase of development:

1. Persistent Database Integration

Currently, all pages read from hardcoded arrays in the `lib/` folder. Changes made inside forms (e.g., creating a course, adding a lecturer, onboarding a school) only show ephemeral toast alerts. The system requires connection to a relational database (such as PostgreSQL) with Prisma ORM to save, update, and delete real institutional records.

2. Server-Side Authentication & Middleware Route Protection

The present authentication gate (`useAuth.ts`) operates in React client-side lifecycle hooks. This allows unauthorized users to temporarily render layout shells before redirects trigger. A Next.js Server Middleware (`middleware.ts`) must be written to read secure session tokens, validate JWTs, and handle server-side redirects before pages reach the browser.

3. Strict Role-Based Access Controls (RBAC)

The app router relies on folder paths to separate roles, but does not strictly validate if a user with lecturer cookies attempts to manually access `/dashboard/super`. Server-side validation logic or Next.js Layout guards must be added to verify that the active user's role exactly matches the target path segment.

4. File Storage Service Integration

The content upload modules are visual placeholders. The next step is integrating cloud object storage (AWS S3, Google Cloud Storage, or Vercel Blob) to support syllabus and lecture slide file uploads.

5. Billing Gateway & Payment Processor Integration

Subscription cards and transactional lists must be backed by a payment gateway API (e.g., Paystack, Flutterwave) to manage institutional renewal billing, webhook events, and automate Tier adjustments.

6. Verification & Local Verification

The project is ready for local verification. Review the guidelines below to execute the development server and test user flows:

Local Installation Commands

Execute these commands in the terminal to initialize dependencies and spin up the project:

```
# Install dependencies
```

```
npm install
```

```
# Spin up the development server
```

```
npm run dev
```

Role Verification Checklist

Log in as one of the following mock accounts to verify specific dashboard behavior:

- Super Admin: Email 'super@rft.edu' | Password 'super123'
Check if the global university management tables, subscriptions, and roles are visible.
- School Admin: Email 'admin@maincampus.edu' | Password 'admin123'
Check if metrics match 'Main Campus' data, and verify lecturers and student levels are active.
- Lecturer: Email 'lecturer@rft.edu' | Password 'lecturer123'
Check course schedule, create assessment constructor, and check performance chart components.

Aesthetic & Visual Guidelines

All components follow a strict design system defined in `tailwind.config.ts` and `app/globals.css`:

- Primary Theme Colors: Deep Navy (#0F1E5A) and Soft Gold (#D4AF37).
- Background: Multi-stage blue-gray gradient background.
- Accent colors are strictly restricted. Text has been kept strictly black, gray, or white according to design requirements (no red texts are permitted; only neutral color scales are used).